五笔打字提示器（电脑版 + 手机版）

开发流程与关键要点

项目	内容
产品名	五笔打字提示器 / 五笔提示器
组成	电脑版（双击 index.html）+ 手机版（APK）
交付目录	E:\5bts deepseeklearn\pc 与 \phone
技术形态	纯前端 HTML/CSS/JS + 本地资源包；手机版用 WebView 壳打包
最终版本	手机版 v1.4（versionCode 5）
当前自检	45 项断言全部通过
文档日期	2026-10-28

本文档记录从需求解析到最终交付的完整开发过程：技术选型理由、离线资源包如何构建、界面与交互如何实现、打包签名怎么做、过程中踩过哪些坑以及如何修掉。可作为后续维护与二次开发的参考手册。

一、总览：一条主线，两个形态

整个产品的核心只有一句话：上面输入，下面输出；汉字在上方标注五笔字母，英文翻译成中文，不认识的语言原样输出并提示；按一下 learn 切换成「千字十篇」的逐字打字练习。

1.1 两种运行形态

形态	位置	运行方式	打包
电脑版	`E:\5bts deepseeklearn\pc\`	双击 index.html，用浏览器打开	免安装，直接拷目录
手机版	`E:\5bts deepseeklearn\phone\`	安装 APK，内部仍是同一套网页	WebView 壳 + 手工 aapt2/d8/apksigner

两个形态共用 90% 以上的代码：界面、逻辑、五个资源包完全一致；手机版额外有一层原生壳（提供返回键、状态栏颜色、联网代发）和一份 mobile.js / mobile.css 做移动端增强。

1.2 完整开发阶段

阶段	产出	关键决策
① 需求解析	从 docx 提取 12 段文字 + 2 张参考图	识别出「翻译按钮改成 learn」等界面要求
② 资源包构建	五笔码表、常用千字表、英汉/汉英词典、十篇文章	全部离线、分目录存放、可独立替换
③ 主程序	index.html + app.css + app.js	纯前端单页，零依赖、可离线双击运行
④ 浏览器验证	无头 Edge 截图 + 自检断言	发现并修掉正则提升、短语吞词等 bug
⑤ 手机版壳	Java Activity + WebView + 手工打包链	不用 Gradle/Android Studio，全程离线
⑥ 迭代增强	夜间模式、联网翻译、额度管理	1.1 → 1.2 → 1.3 → 1.4 逐版累加

二、第一步：需求解析

2.1 从 docx 里挖出可执行的需求

需求文档是一个 Word 文件，正文夹杂两张手机截图。做法：先用 python-docx 把段落、表格、内嵌图片逐个导出到文本文件和独立 jpeg，再逐条翻译成人话。最后得到 6 条可验收的功能点：

- **布局**：上半输入、下半输出；汉字的五笔字母放在汉字正上方（所以字要拉开间距）
- **翻译规则**：英文 → 翻译成中文；其他语言/不明字符 → 原样输出并提示「无法理解」
- **混排**：中英不明字符混排时，各按各的规则分别处理
- **模式切换**：中间那颗「翻译」按钮的位置改成「练习 learn」
- **练习模式**：上半变成「常用 1000 字组成的十篇文章」，下半变成练习输入框
- **练习内功能**：再加两个按钮：答案对照、逐字给出五笔提示；learn 按钮文字随模式变化

需求原文里还有个拼写错误「chiese」，理解为 chinese —— 资源包目录因此命名为 chinese，而不是照抄错字。

2.2 技术选型：为什么是纯前端

候选方案	结论	理由
Electron / Tauri	否	需要 Node 环境，系统没装，且体积大
Python + tkinter	否	能做但界面美化差，中文排版与字间距控制非常麻烦
纯前端网页	✔ 采用	零依赖、双击即开、离线可用、CSS 精确控制字间距与上标码
原生 Android（Kotlin）	仅作壳	只写一个 WebView 壳，界面仍复用网页

关键判断：需求要的是「字上方放五笔字母」，这在 HTML 里就是一个 flex 竖排的 ；要「离线可更新资源包」，就是几个 JS 文件。纯前端几乎以最小成本同时满足了所有约束。

三、离线资源包：五类数据，各管一摊

这是整个项目最费功夫、也最值得复用的部分。原则：所有数据都转成普通 JS 文件，通过 window.registerXxx({...}) 注册，浏览器 <script> 加载即可；要更新某个词表，改对应 .js 文件、刷新页面就行，不用重新编译。

3.1 五类资源包

资源包	文件	规模	来源 / 生成方式
五笔码表	`wubi\wubi86.js`	20,910 字	rime-wubi 的

			wubi86.dict.yaml，单字取最长全码
常用千字	chinese\common1000.js	1,000 字	Jun Da 字频 + jieba 词频双排名融合
英汉词典	english\ecdict_en2cn.js	41,969 词	ECDICT 全量 CSV 精简
汉英反查	english\ecdict_cn2en.js	46,898 词	ECDICT 的 translation 字段反向拆词
英译修正	english\fix_en.js	508 条	手工整理的功能词 + 高频短语表
十篇文章	articles\articles.js	10 篇	用常用千字撰写并校验用字全在千字表内

3.2 五笔码表：单字多码取最长

rime 的码表里一个字常有多个编码（如「我」= q / trn / trnt），全码提示必须取最长那条。规则写死在构建脚本里：

```
# wubi86.dict.yaml（... 之后每行）
#   字 <TAB> 码 <TAB> 权重 [<TAB> 简码]
# 取每个字编码长度最大的那条作为全码
# 输出：上 hhgg / 的 rqyy / 下 ghi（3 码字存在）
```

结果：20,910 个有效汉字，其中 4 码 19,343 个、3 码 1,567 个。

3.3 常用千字：两个字频表取平均名次

只用 jieba 会混进生僻字（涓、锜、襄、郡…），导致「杯、妹、玩、课」这类孩子常用字被挤出前 1000。改成把「Jun Da 现代汉语字频表」和 jieba 排名相加取平均，再用《通用规范汉字表》8105 字过滤：

```
combined[c] = (junda_rank + jieba_rank) / 2   # 缺项按 100000 计
取综合名次最小的前 1000 个字（且必 ∈8105 表）
```

3.4 英汉词典：把 ECDICT 洗成能用的数据

ECDICT 有 77 万行、66 MB，直接塞进浏览器不现实。清洗要点（每一条都是踩坑换来的）：

- **换行：**translation 字段里的换行是「字面量 \n」，必须先 replace 成真换行才能拆义项
- **词性：**词性前缀正则要覆盖 interj./conj. 等，用 (?:[a-zA-Z]{1,8}\.\s*)+
- **去噪：**领域标记 [计] [医] 之类要整行丢弃，否则全是专业噪音

- **排序**：按 frq → bnc → 词长 排序，取前 45000（英汉）/ 25000（汉英）

汉英反查则是把中文释义按标点切成词，只保留 1–6 字纯中文，再记下每个中文词对应哪些英文。这样「表示」能查到 be/show/represent。

3.5 修正表 fix_en.js：纯离线的天花板补丁

离线逐词翻译最大的问题是冠词和功能词（the→那、over→结束、will→意志）。508 条修正表用「最长短语优先」的规则先于词典命中，把整句可读性拉回来：

英文原句	未修正的逐词结果	修正后
`The quick brown fox jumps over the lazy dog.`	那 快 的 褐色 狐狸 跳跃 结束 那 懒惰 的 狗	快的褐色狐狸越过懒惰的狗
`look forward to`	一看 · 向前的 · 到	期待
`go to bed`	去 · 到 · 床	睡觉

修正表是「改一个字就刷新生效」的纯文本，用户自己能加词；这是把离线能力榨到极限、又不引入联网依赖的关键一招。

四、主程序：界面与逻辑

4.1 结构分层

文件	职责
`index.html`	骨架 + 资源包 <script> 加载顺序 + 运行期错误显示框
`assets\app.css`	主题变量、面板布局、「码在上字在下」的格子样式
`assets\app.js`	资源解析、分词、双向翻译、五笔标注、练习判定、自检
`assets\mobile.js`	手机端增强：返回键、软键盘、拼字保护、侧边栏、联网、额度
`assets\mobile.css`	侧边栏、夜间配色、翻译设置面板的样式覆盖

4.2 「码在上、字在下」怎么排

每个字是一个 flex 竖排的小格子：上面 <span.code> 放五笔字母（小号、蓝色、拉开 letter-spacing），下面 <span.ch> 放大字。整段用 flex-wrap 排，标点、英文、空白各是不同的 cell 类型，颜色区分：

```
.cell { display:inline-flex; flex-direction:column; align-items:center; }
.cell > .code { font-size:11px; color:#2f6bff; letter-spacing:2px; }
.cell > .ch   { font-size:26px; }
/* 变体：.cell-punct .cell-foreign(橙) .cell-unknown .cell-net(绿) */
```

需求里「文字要隔开一定距离以便放五笔字母」就是靠格子的 padding 与 gap 实现；标点单独成一格、不显示码，视觉上更清爽。

4.3 输入类型分流（核心算法）

整条输出管线用一个 buildOutput(text, mode, repl) 统一处理，按字符先分词、再分类型：

输入类型	处理	输出样式
汉字	查五笔全码，标注在字上方	蓝码 + 大字
英文	修正表 → 离线词典 →（可选）联网替换	译成中文并标五笔
数字	原样	普通 cell
其他语言（日/韩等）	原样输出	橙色底 + 红条「无法理解」
标点 / 空白	原样，不标码	灰字

英文词形还会做还原（ies/ed/ing/ly/er/est 等），所以 books 能查到 book、running 能查到 run；取义项时先按分号断、再按逗号断，只取第一个，避免把「褐色；褐色的；(使)变褐色」整串吐出来。

4.4 练习模式的状态机

上半显示文章全文（每个字一个格子），下半是输入框。核心是一个 simulate()：把用户输入逐字与原文比对，标出 correct / wrong，标点和空白自动跳过（不要求用户打标点）。

- **两个开关：**「逐字提示」开：每打完一个字，该字上方出现五笔码
- 「答案对照」开：全篇直接显示五笔码，方便对着打
- **模式切换：**learn 按钮本身也是模式开关，按下从日常 → 练习，文字变「↩ 日常 返回」

界面进度显示「已打 N 字 · 错 M」，当前字蓝色高亮并自动 scrollIntoView。

4.5 自检系统：45 条断言

页面带 ?selftest=1 就跑一遍断言，在页面顶部输出 PASS/FAIL 清单。这套自检贯穿整个开发：每改一处逻辑就重跑，保证没把老功能改坏。覆盖分词、双向翻译、词形还原、混排、练习判定、联网替换、额度等。

```
// 示例断言（节选）
check('英文翻译成中文', buildOutput('I love you').stats.enOk === 3);
check('联网替换后中文照常标五笔', ... stats.net === 3);
check('仅标注五笔时不做联网替换', ... stats.net === 0);
```

五、手机版：WebView 壳 + 手工打包

5.1 为什么不用 Gradle

环境里没有 Android Studio、没有 Gradle 缓存，联网下载依赖不确定。于是走最朴素的链路：手工下载 SDK 的 build-tools 与 platform，用 aapt2 / javac / d8 / zipalign / apksigner 六个工具串起来打包，全程离线、一条 build.py 命令出 APK。

```
① aapt2 compile   res\ → .flat
② aapt2 link      生成基础 APK，同时 www\ 铺进 assets\www\
③ javac           编译 Java 源码
④ d8              class → classes.dex
⑤ 塞 dex 进 APK
⑥ zipalign        4 字节对齐
⑦ apksigner       签名（v1/v2/v3）
```

5.2 原生只做三件网页做不到的事

能力	实现	为什么网页做不了
返回键	`onBackPressed` 逐级处理：面板→侧边栏→练习→退出	网页拿不到实体返回键
状态栏/导航栏变色	`NightTask` 在切夜间时改系统栏颜色	浏览器无权改系统栏
联网代发翻译	`NetTask` 后台线程拼签名请求百度，`NetReply` 回灌网页	网页 fetch 受跨域限制

网页与原生之间用 @JavascriptInterface 暴露的 AndroidHost 对象通信；契约很小：getNight/setNight、translate、exit、log。

5.3 联网翻译的三层回退

这是手机版最复杂的一块，设计目标是「任何失败都不能让功能不可用」：

- **第一层**：本地修正表先跑（508 条，0 延迟，常见短句不烧额度）
- **第二层**：剩余英文整段发百度，回来直接替换（队列串行、间隔 1.3 秒守 QPS=1）
- **第三层**：没网/超时/额度用完/密钥错 → 自动退回离线逐词，红条说明原因

另有缓存（同句命中不发请求、不计额度）、分块（单次超 1000 字符按词切）、每月额度统计与上限自动关闭。

六、踩过的坑与修法（最有价值的部分）

问题现象	根因	修法
`Cannot read properties of undefined (reading 'test')`	var 正则声明在文件中部，但开头解析资源时就调用	把所有正则与 isCJK 上移到工具区
短语命中后吞掉后面的词（with you 消失）	gatherWords 返回的是「收集到的最后一个词」而非「短语实际长度」	返回 idxs 数组，只跳过多词短语自身占用的词
中→英输出「I love North 京 day 安 door」	反查词典缺北京/天安门/安/京，家首选 move	新增 ZH_OVERRIDE 约 70 条高频固定译法，先于词典命中
抽屉在无头/后台环境打不开	requestAnimationFrame 在后台 WebView 不执行	void mask.offsetWidth 强制重排后同步加 class
改中文文件后全文乱码	PowerShell ReadAllText 按 GBK 读 UTF-8	改用 Python 显式 encoding='utf-8'，绝不用 PS 读写中文
凭空报 52003 但密钥没错	把「应用名称」当成了 APP ID	APP ID 必须纯数字长串，密钥 16 位字母数字

七、验证方法：截图 + 断言 + 真机

- **浏览器**：用无头 Edge（不是 Chrome，Chrome 报 ICU 错）加 --allow-file-access-from-files 截图与 dump-dom，中文 URL 参数先 urllib.quote 预编码
- **自检**：?selftest=1 跑 45 条断言，正则抽取页面上的 PASS/FAIL 清单
- **真机**：雷电模拟器 adb install，用 am start 带 -e query '?selftest=1' 看 logcat 里的自检结果

- **打包**：aapt dump badging 核对版本号；check_apk.py 逐条核对 APK 内 29 个条目

经验：截图 + 断言 + 真机三层验证缺一不可 —— 断言保逻辑、截图保观感、真机保环境差异（比如 rAF 在后台不跑这种只有真机会暴露的问题）。

八、版本演进

版本	主要内容
1.0	日常模式 + 练习模式、五笔标注、英汉词典、千字十篇
1.1	左侧边栏 + 夜间模式（设置会记住）；拼音拼字不再误判；小屏输出区限高
1.2	新增 fix_en.js 修正表（508 条）：冠词不输出、功能词改对、短语整段替换；自检 25→38
1.3	联网翻译开关 + 翻译设置面板：百度 API + 原生代发，失败回退离线；自检 38→45
1.4	安全加固（密钥圆点显示、不进备份、远程调试默认关）+ 每月额度上限与实时用量

九、可复用的方法论

- **需求**：需求先翻译成「可验收的功能点」，再动手；模糊处（如 chiese）按意图理解并记录
- **资源**：数据全部转普通 JS 文件 + register 函数，做到「改文件即更新、零编译」
- **架构**：纯前端能覆盖 90% 需求时就不要引入重框架；原生只补网页做不到的那三件事
- **质量**：每一版都带一套自检断言，改代码先跑断言，防止回归
- **验证**：截图看观感、断言看逻辑、真机看环境，三者互补
- **健壮**：联网一律三层回退，任何外部依赖失效都要能优雅降级到本地